

Stem Agent: Blueprint-Based Specialisation with Tool Feedback

Vadim Pankov

1 Introduction

I interpreted the stem-agent task as controlled specialisation rather than unrestricted self-rewriting. The system starts from a broad task-family prompt, classifies the task using an LLM-powered classifier, profiles required capabilities, generates a specialist blueprint, evolves it through a multi-round mutation loop, evaluates both structural and task-level quality using real tool results, and executes the selected specialist.

The central question the project addresses is: does a better blueprint actually produce a better agent? The answer, demonstrated across three benchmarks, is yes—but the mechanism matters. Blueprint structure alone does not improve output quality. What matters is whether the blueprint's architecture enables a real tool feedback loop. The mutated blueprint does; the base blueprint does not.

2 Stem-Agent Pipeline

2.1 Stem Shell: Initial Differentiation

The StemShell receives a task description, delegates domain analysis to the DomainProfiler, and produces an initial AgentBlueprint as a structured JSON configuration containing the proposed role, workflow, tools, output schema, and stopping conditions. At this point the system does not execute the task; it performs the first controlled step of differentiation.

2.2 Classifier: Reading the Task Environment

The DomainClassifier reads the task prompt semantically rather than matching against keyword lists. When an API key is available it sends the raw prompt to the LLM with a system prompt constraining the output to a structured JSON object containing domain, subdomain, intent, confidence, and a one-sentence reasoning field. The LLM proposes the classification from scratch; it does not pick from a menu presented in the prompt. A keyword fallback is retained for the deterministic path (no API key) and provides a measurable baseline.

Across five benchmark tasks, the LLM classifier was used in all five cases and agreed with the keyword classifier in all five. The reasoning field—for example, 'The task explicitly requests adding comments and docstrings, which falls under quality assurance'—is preserved in the DomainProfile and surfaced in both the CLI and Streamlit frontend, making the routing decision inspectable rather than implicit.

The remaining limitation is that the five domain categories are still predefined. The LLM reads the environment semantically, but routes into a fixed template set. This is acknowledged in §7.2.

2.3 Architecture Generator

The ArchitectureGenerator converts the domain profile into an initial specialist blueprint. A debugging profile becomes a `debugging_specialist_agent` with workflow steps for reading source code, inferring expected behaviour, identifying failure symptoms, generating tests, localising the bug, suggesting a fix, and producing a report. Security and research profiles are similarly converted into specialised architectures.

3 Blueprint Mutation and Evaluation

The BlueprintMutator modifies safe configuration fields: workflow steps, output schema, role description, and stopping conditions. Source code is never modified. For debugging blueprints, mutation adds active verification steps such as `verify_fix_against_generated_tests` and `check_for_regression_risk`. For documentation blueprints, mutation adds `verify_docstring_coverage` as an active step that triggers the

quality-checking feedback loop.

The BlueprintEvaluator uses a two-layer scoring scheme: structural quality at 40% weight and task-level quality at 60% weight. Task-level scoring uses real tool results—pytest pass rate for debugging, quality-aware docstring coverage for documentation, secret scan findings for security. The MutationLoop runs for up to four rounds and stops when the combined score improvement falls below 3%, or when tool-verified quality meets the domain target.

3.1 Stopping Criterion Design

The system has two distinct stopping problems. For blueprint evolution, the loop stops when task-level evidence stops improving rather than when structural self-assessment plateaus—this breaks the circular dependency where mutation adds exactly the safeguards the evaluator rewards. The 3% threshold applies to a score driven primarily by tool results, not self-defined criteria. For execution, the debugging agent stops when pytest passes (external, objective signal) or when MAX_REVISION_ROUNDS is exhausted (cost-safety ceiling). The documentation agent stops when quality coverage reaches 90% as measured by the AST-based checker, or after two revision rounds. In both cases the agent stops because a real tool confirmed the quality target was met, not because it decided it was done.

4 Execution Layer

The deterministic AgentRunner makes the blueprint executable without an API key. Five tools are registered in a TOOL_REGISTRY: pytest_runner, static_checker, complexity_checker, secret_detector, and docstring_checker. Tool results feed directly into the evaluator's task-level score.

The LLM-backed runner implements tool feedback loops for two domains. For debugging, after the initial fix attempt it calls pytest_runner and sends failure output back to the LLM as a new conversation turn, repeating for up to three rounds. For documentation, after the initial pass it calls the quality-aware docstring checker—a symbol counts as complete only if it has Args: and Returns: sections where required—and sends back the list of incomplete symbols for targeted revision, repeating for up to two rounds. The distinction between passive steps (run_tests_if_available) and active steps (verify_fix_against_generated_tests, verify_docstring_coverage) determines whether the loop is triggered: only active steps, added by mutation, enable the feedback loop. Both blueprints use the same model, so any performance difference is attributable to blueprint architecture.

5 Benchmarks

5.1 Blueprint Structural Benchmark

Across all five task categories the base blueprint scored 0.86 combined and the mutated blueprint scored 1.00; the mutated blueprint was selected in every case. This validates the structural differentiation loop but does not prove real task-solving performance.

5.2 Debugging Ablation (30 Self-Curated Tasks)

Both blueprints were run against 30 hand-curated Python functions across six bug categories using the same LLM runner. The agent receives only the buggy code and never sees the ground-truth fix.

Metric	Base Blueprint	Selected Blueprint
Fix extracted rate	100%	100%
First-attempt pass rate	73%	70%
Final pytest pass rate	73%	97%
Avg revision rounds	0.00	0.43

Avg agent score	0.813	0.977
-----------------	-------	-------

Table 1: 30-task debugging ablation. Base: single-shot. Selected: tool feedback loop.

5.3 QuixBugs External Validation (10 Real Bugs)

To address the self-curation concern, the system was validated on 10 bugs from the QuixBugs dataset (Koppel et al.), an independent collection of real algorithm bugs not seen during development. The benchmark was run twice to assess stability under LLM variance.

Blueprint	Run 1 pass rate	Run 2 pass rate
Base (single-shot)	0/10 (0%)	0/10 (0%)
Mutated (feedback loop)	7/10 (70%)	5/10 (50%)

Table 2: QuixBugs external validation. Base: 0% both runs. Mutated: 50–70%. Variance reflects LLM non-determinism; direction is consistent.

5.4 Documentation Feedback Loop (10 Multi-Function Modules)

A quality-aware docstring checker was built using Python’s ast module. A docstring counts as complete only if it has an Args: section for functions with parameters and a Returns: section for functions with a return value. One-liner docstrings score zero. Ten multi-function Python modules (4–6 public symbols each) were used as tasks.

Blueprint	Avg quality coverage	Improved tasks	Avg rounds
Base (single-shot)	96%	—	0.00
Mutated (feedback loop)	100%	2/10	0.20

Table 3: Documentation benchmark. In 2/10 tasks the checker found coverage below 90% after round 0 and fired a revision round.

5.5 Classifier Comparison

Across all five benchmark tasks the LLM classifier was used and agreed with the keyword classifier in every case. The agreement rate validates that the keyword signals were well-chosen for these categories. The LLM classifier adds value on novel or ambiguous phrasing outside the keyword vocabulary—a limitation of the current benchmark set rather than the classifier itself.

Benchmark	Before	After
Blueprint structural score	0.86	1.00
Deterministic vs LLM runner	0.60	0.95
30-task ablation (final pytest)	73%	97%
QuixBugs external (mutated)	—	50–70%
Doc quality coverage (mutated)	—	100%

Table 4: Summary of all before/after comparisons.

6 Conclusion

6.1 What Surprised Me

The first-attempt regression of –3% in the debugging benchmark was the most informative result. The verification prompt makes the model slightly more cautious on the first attempt. This was only visible because first-attempt and final pass rates were measured separately. It is also the clearest evidence that

blueprint architecture is causally responsible for the performance gap: the selected blueprint starts slightly worse and recovers through the feedback loop, while the base plateaus at 73%.

The documentation result revealed a subtler version of the same problem. Presence-only coverage measurement (does any docstring string exist?) always returns 100% because the LLM adds a string to every symbol in one pass. Quality-aware measurement (does the docstring have Args: and Returns: sections where required?) exposes the real gap that the feedback loop can act on. The checker design mattered as much as the loop design.

The QuixBugs result—0% base, 50–70% mutated across two runs—was stronger than expected. The base blueprint's single-shot approach consistently failed on all 10 real bugs, while the feedback loop recovered the majority. The variance between runs is honest: LLM non-determinism affects absolute numbers but not the direction of improvement.

6.2 What Failed or Did Not Work Well

The most important limitation is that the five domain categories remain predefined. The LLM classifier reads the task environment semantically, which is a genuine improvement over keyword lists, but it still routes into fixed templates. A true stem agent would discover its own categories. The system is closer to a configurable code-agent factory with emergent environment-reading than to a fully self-discovering stem mechanism.

The documentation feedback loop produced modest but real results: 2/10 tasks improved. The cases where base already achieved 100% quality coverage showed that for simpler modules the model is already thorough enough that the loop adds no value. The loop only matters when there is a genuine gap—which requires a strict enough metric to detect it.

Type/scope errors remain at 80% final pass rate even after revision: mutable default arguments and global scope bugs require understanding Python's execution model in ways that pytest output alone does not make sufficiently explicit. Three QuixBugs tasks (`is_valid_parenthesization`, `next_palindrome`, `possible_change`) hit `MAX_REVISION_ROUNDS` without passing—the feedback loop ran out of budget before finding the fix.

6.3 What I Would Do With More Time

With more time I would extend the feedback loop to security (`secret_detector` findings → mitigate iteratively) and code-quality (`complexity_checker` output → targeted refactor). I would replace the predefined domain templates with LLM-proposed blueprint architectures: given a domain profile, ask the LLM to generate candidate workflow steps and tools from scratch, then evaluate and select deterministically. This would make specialisation genuinely discovered rather than selected from a predefined set. I would also run the full 40-task QuixBugs suite and evaluate on BugsInPy for a larger real-world validation.

Repository: github.com/Vadim3377/AI-agent